

Laporan Praktikum Kontrol Cerdas

Minggu Ke-2

Nama : Rani Dwi Setiani
NIM : 224308043
Kelas : TKA-6B
Akun Github : <https://github.com/Rani-TKA6B>
Student Lab Assistant : Mas Alfian

1. Judul Percobaan: *Object Classification with Scikit-learn*
2. Tujuan Percobaan:
 - Memahami dasar-dasar *Machine Learning* dalam sistem kendali.
 - Mengimplementasikan model ML sederhana untuk klasifikasi objek.
 - Menggunakan Scikit-learn untuk membuat model ML dasar.
 - Mengintegrasikan model ML dengan Computer Vision untuk deteksi objek.
 - Mengelola dataset dan melakukan pelatihan model sederhana.
3. Landasan Teori:

Machine Learning adalah salah satu bidang cabang ilmu komputer yang memberikan kemampuan kepada komputer untuk dapat belajar tanpa diprogram secara eksplisit (Daqiqil, 2021). Untuk bisa mengaplikasikan teknik-teknik machine learning maka harus ada data. Tanpa data maka algoritma machine learning tidak dapat bekerja. Data yang ada biasanya dibagi menjadi dua, yaitu data training dan data testing. Data training digunakan untuk melatih algoritma, sedangkan data *testing* digunakan untuk mengetahui performa algoritma yang telah dilatih sebelumnya ketika menemukan data baru yang belum pernah dilihat. *Machine Learning* sendiri terbagi menjadi beberapa algoritma yang berbeda-beda dan setiap dari algoritma tersebut memiliki fungsi dan tujuannya masing-masing, seperti *supervised learning*, *unsupervised learning*, dan *reinforcement learning* (Karimah, 2023). *Supervised Learning* adalah algoritma yang menyimpulkan fungsi dari data pelatihan berlabel yang terdiri dari sekumpulan contoh pelatihan, misalnya seperti *logistic regression* dan *K-Nearest Neighbors* (KNN). *Unsupervised Learning* adalah algoritma

yang bertujuan untuk meningkatkan kinerja tugas pembelajaran yang diawasi dimana kita tidak memiliki banyak data, contohnya seperti *k-means*, *Apriori*, DBSCAN, dan *clustering*. *Reinforcement Learning* adalah algoritma yang mengumpulkan pengetahuan (sinyal penguatan) untuk memilih tindakan yang mengarah ke hasil tertinggi yang diharapkan. Dibandingkan dengan metodologi lain dalam *Machine Learning*, algoritma *Reinforcement Learning* memiliki kelebihan yang berbeda yaitu kemampuan untuk secara mandiri menjelajahi lingkungan yang sangat dinamis dan stokastik dan mengembangkan, dengan mengumpulkan umpan balik evaluatif dari lingkungan, kebijakan kontrol yang optimal (Fathurohman, 2021). *Machine Learning* memungkinkan sistem kendali untuk belajar dari data dan meningkatkan performanya seiring waktu. Beberapa contoh penerapannya yaitu *Predictive Maintenance* (mendeteksi potensi kegagalan sistem sebelum terjadi), *Adaptive Control* (sistem kendali yang dapat menyesuaikan parameter secara otomatis), dan *Anomaly Detection* (mendeteksi perilaku tidak normal dalam sistem industri) (Roihan dkk., 2020).

4. Analisis dan Diskusi:

- Analisis

Percobaan praktikum pada minggu kedua ini adalah memodifikasi program menggunakan model *Machine Learning* lain yaitu ***Decision Tree*** dengan menambahkan fitur mendeteksi lebih dari satu warna secara simultan. *Decision Tree* mampu mengklasifikasikan warna dengan baik berdasarkan fitur warna RGB, sehingga dapat mengklasifikasikan warna dengan cepat dan mudah diinterpretasikan. Percobaan ini bertujuan untuk meningkatkan akurasi dan fleksibilitas sistem dalam mengenali warna pada suatu objek. ***Decision Tree*** adalah salah satu algoritma *Machine Learning* berbasis pohon keputusan yang digunakan untuk klasifikasi dan regresi. Algoritma ini bekerja dengan membagi dataset ke dalam subset yang lebih kecil berdasarkan fitur-fitur tertentu. Keunggulan *Decision Tree* adalah tidak memerlukan banyak preprocessing dibanding model lain seperti SVM, Mudah diinterpretasikan karena berbentuk aturan keputusan, Cepat dalam melakukan prediksi karena tidak memerlukan perhitungan

kompleks seperti model berbasis kernel. Dibandingkan dengan metode sebelumnya, Decision Tree memberikan kecepatan prediksi yang lebih baik dalam beberapa kasus, meskipun masih memiliki keterbatasan dalam menangani variasi pencahayaan yang ekstrem. Penambahan fitur deteksi lebih dari satu warna memungkinkan sistem untuk mengenali berbagai warna secara bersamaan dalam satu gambar. Hal ini meningkatkan fleksibilitas sistem dalam pengenalan objek yang memiliki variasi warna kompleks. Penyesuaian threshold pada model warna dapat membantu meningkatkan akurasi lebih lanjut. Kelemahan *Decision Tree* yaitu Rentan terhadap *overfitting*, terutama jika terlalu banyak node dalam pohon keputusan, Kurang optimal dalam mendeteksi warna yang mirip, misalnya merah terang dan oranye dapat diklasifikasikan secara keliru, Sensitif terhadap variasi pencahayaan, sehingga warna yang terlihat berubah dapat menyebabkan kesalahan prediksi. Jika jumlah dataset ditambah, model *Decision Tree* dapat menunjukkan perubahan akurasi dapat mempelajari pola yang lebih baik, dan Akurasi meningkat karena model lebih generalized dan tidak hanya menghafal pola tertentu. Tetapi jika dataset mengandung *noise* atau tidak seimbang, Model bisa menjadi *overfitting*, terutama jika dataset memiliki banyak variasi tetapi tidak cukup contoh per warna, Akurasi dapat menurun jika model tidak dapat membedakan warna dengan baik dalam kondisi pencahayaan berbeda. Jika warna akurasi tepat dan seimbang dapat dikerjakan dengan menyeimbangkan dataset dengan jumlah warna yang merata, Menambahkan data dari berbagai sumber pencahayaan agar model lebih *robust*, Menggunakan teknik data *augmentation*, seperti mengubah kecerahan atau saturasi warna saat pelatihan. Untuk meningkatkan kinerja *Decision Tree*, beberapa metode dapat diterapkan yaitu dengan Pruning (**max_depth** atau **min_samples_split** agar pohon tidak terlalu dalam), Menggunakan Model Ensemble (*Random Forest*), *Feature Engineering* dan Normalisasi Data (Menggunakan representasi warna yang lebih baik seperti HSV dibanding RGB, dan Menstandarisasi data menggunakan StandardScaler agar setiap fitur memiliki distribusi yang seragam), Meningkatkan Kualitas Dataset,

Fine-Tuning Hyperparameter (Menggunakan teknik *Grid Search* untuk mencari kombinasi terbaik).

- Diskusi

Metode berbasis aturan (*rule-based*) bekerja dengan aturan yang ditentukan secara eksplisit, sedangkan *Machine Learning* (ML) belajar pola dari data tanpa aturan yang ditentukan manual. Lebih fleksibel dapat menangani data yang kompleks dan dinamis tanpa perlu menulis aturan secara eksplisit. Mampu mengenali pola tersembunyi dengan menemukan hubungan yang tidak jelas dalam data, yang sulit dideteksi oleh aturan manual. Skalabilitas lebih tinggi bisa diterapkan pada data dalam skala besar tanpa perlu menulis ulang aturan. Lebih adaptif dapat terus belajar dan memperbaiki kinerjanya seiring bertambahnya data. *Machine Learning* dapat diintegrasikan lebih lanjut dalam sistem kendali otomatis. Seperti Sistem kendali berbasis ML dapat memproses data dari kamera, radar, atau sensor lain untuk membuat keputusan, contohnya Drone otomatis yang mengenali rintangan dan menyesuaikan jalur terbangnya. Meskipun ML memiliki banyak keunggulan, ada beberapa tantangan dalam penerapan *real-time*, Seperti Model ML harus memberikan prediksi dalam hitungan milidetik agar bisa digunakan dalam sistem real-time seperti kendaraan otonom atau robot industri, dengan memaksimalkan Optimasi model dengan *quantization*, *pruning*, atau model *lightweight* seperti TinyML. Pada program yang dipakai ini memanfaatkan *Decision tree* mendeteksi warna dalam pemrosesan citra dilakukan dengan memanfaatkan representasi warna dalam model warna seperti RGB atau HSV. Dengan menggunakan model HSV, rentang warna yang lebih fleksibel dapat ditentukan, sehingga sistem dapat mengenali berbagai warna secara lebih akurat. Model *Decision Tree* dapat dikonfigurasi dengan baik untuk tugas klasifikasi warna, dengan pembagian dataset yang optimal, sistem mampu mengenali warna dengan akurasi yang cukup tinggi. Dibandingkan dengan metode sebelumnya, *Decision Tree* memberikan kecepatan prediksi yang lebih baik dalam beberapa kasus,

meskipun masih memiliki keterbatasan dalam menangani variasi pencahayaan yang ekstrem.

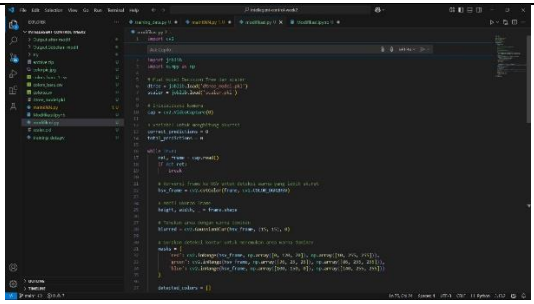
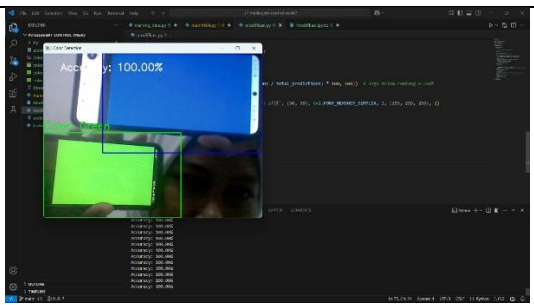
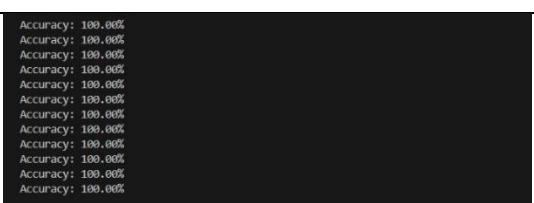
5. *Assignment:*

Program ini melakukan sejumlah tugas (penugasan variabel) untuk menyimpan data dan konfigurasi penting yang diperlukan dalam deteksi warna. Pertama, Dataset warna dikumpulkan dalam bentuk nilai RGB atau HSV, Agar model bekerja optimal, fitur dalam dataset distandarisasi menggunakan **StandardScaler()**, ini membantu model untuk berkonvergensi lebih cepat dan meningkatkan akurasi prediksi. Setiap data warna diberi label sesuai dengan klasifikasinya ('R', 'G', 'B'). Model *Decision Tree* dilatih menggunakan dataset yang telah diproses. Model disimpan dalam file **dtree_model.pkl** untuk digunakan dalam deteksi real-time. Selanjutnya membuat model dan scaler **dtree = joblib.load('dtree_model.pkl')** **scaler = joblib.load('scaler.pkl')** Model *Decision Tree* yang telah dilatih sebelumnya dimuat dari **file**. **StandardScaler** juga dimuat untuk memastikan data input memiliki skala yang sama. Selanjutnya inisiasi kamera **cap = cv2.VideoCapture(0)** kamera diaktifkan untuk menangkap frame secara real-time. Pada setiap frame mendeteksi warna dari kamera dengan konversi warna **hsv_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)** gambar diubah dari format BGR ke HSV untuk meningkatkan akurasi deteksi warna, dengan Gaussian blur dapat diterapkan untuk mengurangi noise pada gambar sehingga warna mudah untuk dikenali **blurred = cv2.GaussianBlur(hsv_frame, (15, 15), 0)**. Menampilkan warna yang terdeteksi dengan kotak yang dibuat disekitar area warna yang terdeteksi, dan nama warna ditampilkan pada layer. Terdapat akurasi di layar, dihitung berdasarkan jumlah deteksi warna yang sesuai dengan ekspektasi

Hasil prediksi serta akurasi ditampilkan di layar kamera dengan informasi yang diperbarui secara dinamis berdasarkan *running accuracy* dari 50 prediksi terakhir. Penugasan variabel yang terstruktur dengan baik dalam program ini memungkinkan aliran data yang lancar dari *preprocessing*, *training model*, hingga prediksi *real-time*.

6. Data dan Output Hasil Pengamatan

[illegible]

Sesudah Modifikasi		
2.	<i>Coding</i> sesudah modifikasi dengan menggunakan model <i>Machine Learning</i> lain yaitu <i>Decision tree</i>, dan dengan menambahkan fitur untuk mendeteksi lebih dari satu warna secara simultan, serta menambahkan dataset lain pada program.	
	<i>Running</i> secara <i>real-time</i> dan memunculkan warna <i>green</i> dan <i>blue</i> secara bersamaan dan memunculkan <i>color</i> yang terdeteksi pada kamera	
	Presentase akurasi secara <i>real-time</i> : 100%	

7. Kesimpulan:

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

- Program ini untuk mendeteksi warna dengan menggunakan *Decision Tree*.
- Menggunakan *StandardScaler* membantu model untuk berkonvergensi lebih cepat dan meningkatkan akurasi prediksi.
- *Running accuracy* memberikan evaluasi hasil deteksi model yang lebih akurat dan dinamis.

- Hasil prediksi dan keakuratan objek warna bergantung pada pencahayaan dan kualitas kamera.

8. Saran:

Untuk meningkatkan akurasi yang tepat, terdapat beberapa hal yang dapat dilakukan yaitu di antaranya, menggunakan *color correction* pada tahap *processing*. Kemudian, agar agar model lebih adaptif terhadap perubahan pencahayaan dengan menambahkan *adaptive thresholding*. Selain itu, perluas dataset warna dengan lebih banyak variasi untuk meningkatkan kemampuan generalisasi model dalam mengenali warna lebih akurat.

9. Daftar Pustaka:

- Daqiqil, I. (2021). *Machine Learning: Teori, Studi Kasus, dan Implementasi Menggunakan Python*. Riau: UR PRESS.
- Fathurohman, A. (2021). Machine Learning untuk Pendidikan: Mengapa dan Bagaimana. *1*(3).
- Karimah Tauhid, Volume 2 Nomor 1 (2023), e-ISSN 2963-590X. (2023). 2.
- Roihan, A., Sunarya, P. A., & Rafika, A. S. (2020). Pemanfaatan Machine Learning dalam Berbagai Bidang: Review paper. *IJCIT (Indonesian Journal on Computer and Information Technology)*, 5(1). <https://doi.org/10.31294/ijcit.v5i1.7951>